

PsycheNavigator Backend

Detailed Code Quality Review & Improvement Plan

Scope: all backend source files (API, agent graph, nodes, RAG pipeline, utilities)
Date: July 6, 2026

Severity	Count	Meaning
HIGH	7	Can crash the server, silently break retrieval, or hang the frontend
MEDIUM	12	Maintainability risks, fragile logic, hidden failures, missing deps
LOW	9	Style, consistency, and polish

Executive Summary

The backend is well organized: clear separation between the API layer (FastAPI), the agent orchestration (LangGraph), the individual reasoning nodes, and the RAG pipeline (ChromaDB + local embeddings). Docstrings are unusually thorough, the ingestion junk-filter is a nice touch, and secrets hygiene is correct (.env is gitignored, .env.example is tracked).

The most impactful problems fall into four themes: **(1) missing input validation and error handling** on the API surface — an empty message list crashes /chat, and any LLM failure mid-stream kills the SSE connection with no error event, leaving the frontend hanging; **(2) fragile startup wiring** — every module creates LLM clients, embeddings, and the vectorstore at import time using relative paths, so starting the server from the wrong directory silently creates an *empty* vector database and retrieval returns nothing; **(3) LangGraph anti-patterns** — nodes mutate the shared state in place and return the full state instead of partial updates; **(4) duplication** — the book catalog lives in four places, the max-retry constant in two, and the history-to-LangChain-messages conversion in two.

None of these require an architecture change. Each finding below includes the exact file and line, why it matters, and a concrete fix. A suggested priority order is at the end of the document.

Files Reviewed

- api/main.py — FastAPI app, SSE chat endpoint, DB exploration endpoints
- agent/graph.py — LangGraph wiring and routing
- agent/state.py — AgentState TypedDict
- agent/logger.py — ANSI terminal logging helpers
- agent/nodes/guard.py — on/off-topic classifier
- agent/nodes/retriever.py — ChromaDB search + chunk filtering
- agent/nodes/evaluator.py — chunk sufficiency judge
- agent/nodes/query_builder.py — query rephrasing on retry
- agent/nodes/psychologist.py — grounded advice generation
- agent/nodes/action_planner.py — structured action plan
- agent/nodes/follow_up.py — follow-up question generation
- rag/ingest.py — PDF ingestion pipeline
- rag/retriever.py — vectorstore + retriever construction
- llm_factory.py — Groq LLM factory
- start.py — entry point

High-Severity Findings

[HIGH] 1. Empty or malformed chat request crashes the endpoint

api/main.py:63-66

Problem: POST /chat with {"messages": []} raises an IndexError on recent_messages[-1] and returns an unhandled 500. There is also no check that the last message has role "user" or non-empty content.

Why it matters: Any frontend bug, retry logic, or manual API call with an empty history takes down the request with a stack trace instead of a clean 422. Public-facing endpoints should never 500 on bad input.

Recommended fix: Enforce non-empty messages at the schema level so FastAPI rejects it before your code runs:

```
from pydantic import BaseModel, Field
from typing import List, Literal

class Message(BaseModel):
    role: Literal["user", "assistant", "ai"]
    content: str = Field(min_length=1)

class ChatRequest(BaseModel):
    messages: List[Message] = Field(min_length=1)
```

[HIGH] 2. No error handling in the SSE stream — LLM failures hang the frontend

api/main.py:80-95 (stream generator)

Problem: If any node raises mid-stream (Groq rate limit, network timeout, ChromaDB error), the async generator dies. The SSE connection just stops: no error event, no __end__ event.

Why it matters: The frontend never learns the request failed — it will spin forever waiting for the next event. Groq rate limits make this a when, not an if.

Recommended fix: Wrap the stream body in try/except and emit a structured error event so the UI can recover:

```
async def stream():
    try:
        async for graph_output in graph.astream(initial_state):
            for node_name, state_update in graph_output.items():
                ...
                yield f"data: {json.dumps(...)}\n\n"
    except Exception as exc:
        yield f"data: {json.dumps({'node': '__error__', 'error': str(exc)})}\n\n"
    finally:
        yield f"data: {\"node\": \"__end__\"}\n\n"
```

[HIGH] 3. Relative paths make the vector DB location depend on the launch directory

rag/retriever.py:10 ('./chroma_db'), rag/ingest.py:108, rag/ingest.py:59-66 ('books/...')

Problem: Chroma(persist_directory="./chroma_db") resolves against the current working directory. If the server is started from the repo root instead of backend/ (e.g. uvicorn backend.api.main:app, an IDE run config, or Docker), Chroma silently creates a brand-new empty database and every retrieval returns zero chunks — no error, just an agent that always fails evaluation.

Why it matters: This is the most dangerous kind of bug: everything appears to work, but retrieval quality is mysteriously zero. It also makes deployment fragile.

Recommended fix: Anchor all paths to the source file, and reuse the constant in both ingest and retriever:

```
# backend/paths.py (or config.py)
from pathlib import Path
BACKEND_DIR = Path(__file__).resolve().parent
CHROMA_DIR = BACKEND_DIR / "chroma_db"
BOOKS_DIR = BACKEND_DIR / "books"

# rag/retriever.py
_vectorstore = Chroma(persist_directory=str(CHROMA_DIR), ...)
```

[HIGH] 4. Heavy side effects at import time (LLM clients, embeddings, vectorstore)

guard.py:6, evaluator.py:6, query_builder.py:6, psychologist.py:5, action_planner.py:7, follow_up.py:5, rag/retriever.py:4-15, api/main.py:19

Problem: Every node module calls make_llm() at import, and rag/retriever.py loads the HuggingFace embedding model and opens ChromaDB the moment it is imported. If GROQ_API_KEY is missing, the app dies during import with a confusing traceback deep inside module loading. load_dotenv() only runs in start.py, so launching with plain uvicorn api.main:app skips .env entirely and crashes.

Why it matters: Import-time side effects make the app impossible to unit-test (importing a node requires a real API key and a 90 MB embedding model), slow down every tool that touches the code, and couple startup order to import order.

Recommended fix: Call load_dotenv() in api/main.py too (it is idempotent), and defer construction with a cached factory instead of module-level globals:

```
# llm_factory.py
from functools import lru_cache

@lru_cache(maxsize=None)
def get_llm(temperature: float = 0.0) -> ChatGroq:
    api_key = os.getenv("GROQ_API_KEY")
    if not api_key:
        raise RuntimeError("No GROQ_API_KEY found in environment")
    return ChatGroq(model=MODEL, temperature=temperature, groq_api_key=api_key)

# in a node: llm = get_llm(0.3) # inside run(), not at module level
```

[HIGH] 5. LangGraph nodes mutate state in place and return the full state

All node `run()` functions (`guard.py:27-66`, `retriever.py:74-101`, `evaluator.py:56-111`, `query_builder.py:22-44`, `psychologist.py:64-92`, `action_planner.py:35-57`, `follow_up.py:24-51`)

Problem: Nodes do `state["x"] = ...; return state`. LangGraph's contract is that a node returns a *partial update dict*; the framework merges it. Mutating the input and returning everything works today, but it breaks state-diffing: every streamed update in `/chat` contains all 13 keys, so the frontend cannot tell what a node actually changed, and adding a checkpoint or parallel branches later will produce subtle bugs.

Why it matters: Returning partial updates makes each node's contract explicit, shrinks SSE payloads, and is the documented LangGraph pattern.

Recommended fix: Return only the keys the node changed:

```
def run(state: AgentState) -> dict:
    ...
    if "off_topic" in classification:
        return {"is_off_topic": True,
                "final_response": _OFF_TOPIC_MESSAGE,
                "action_plan": [], "active_frameworks": []}
    return {"is_off_topic": False,
            "current_query": user_message, "retrieval_attempts": 0}
```

[HIGH] 6. Re-running ingestion crashes; chunk IDs are not stable

`rag/ingest.py:109-112`, `rag/ingest.py:116`

Problem: `client.create_collection(name="langchain")` raises if the collection already exists, so `ingest.py` can only ever be run once against a given `chroma_db` directory. Chunk IDs are `str(index)`, which means any partial re-ingest would silently overwrite unrelated chunks.

Why it matters: Re-ingesting (new book, changed chunking, fixed filter) is a normal workflow; today it requires manually deleting the directory, and if someone switches to `get_or_create` the index-based IDs would corrupt the collection.

Recommended fix: Recreate the collection explicitly and derive IDs from content position:

```
try:
    client.delete_collection("langchain")
except Exception:
    pass
collection = client.create_collection(
    name="langchain", metadata={"hnsw:space": "cosine"})

chunk_ids = [f"{c.metadata['source']}-p{c.metadata.get('page', 0)}-{i}"
              for i, c in enumerate(all_chunks)]
```

[HIGH] 7. Blocking LLM and DB calls inside an async streaming endpoint

All nodes use synchronous `.invoke()`; `api/main.py:88` uses `graph.astream`

Problem: Each node performs a blocking network call to Groq. LangGraph runs sync nodes in a thread pool so the event loop survives, but every concurrent `/chat` request pins a worker thread for the whole multi-node pipeline (often 10-30 seconds), and the thread pool is a hard concurrency cap.

Why it matters: Under even modest concurrent load, requests queue behind the thread pool. The fix is cheap because LangChain runnables already expose async variants.

Recommended fix: Make node `run()` functions async and await the chain:

```
async def run(state: AgentState) -> dict:
    response = await (prompt | llm).ainvoke({"user_message": user_message})
    ...
```

Medium-Severity Findings

[MEDIUM] 8. requirements.txt is missing a directly-imported package

`requirements.txt` vs `rag/retriever.py:1`

Problem: `rag/retriever.py` imports `langchain_chroma`, but `langchain-chroma` is not in `requirements.txt`. It currently works only because some other package pulled it in. `langchain-text-splitters` (imported in `ingest.py`) is likewise only a transitive dependency.

Why it matters: A fresh `pip install -r requirements.txt` on another machine (or after an upstream dependency bump) can fail with `ImportError` at startup.

Recommended fix: Declare every directly-imported package: add `langchain-chroma>=0.1.0` and `langchain-text-splitters>=0.2.0`.

[MEDIUM] 9. The book catalog is duplicated in four places

`rag/ingest.py:59-66 (BOOKS)`, `agent/nodes/retriever.py:7-14 (_KNOWN_BOOKS)`, `agent/nodes/evaluator.py:38-44 (prompt list)`, `agent/nodes/psychologist.py:56-57 (prompt list)`

Problem: The six book titles are hardcoded independently in the ingestion map, the retriever filter, the evaluator prompt, and the psychologist citation whitelist.

Why it matters: Adding or renaming a book requires touching four files; missing one produces silent quality bugs (e.g. the retriever filter drops every chunk of a newly ingested book because `_KNOWN_BOOKS` was not updated — the exact silent-empty-retrieval failure mode again).

Recommended fix: Create a single `books.py` (or `config.py`) exporting the catalog; derive `_KNOWN_BOOKS`, the ingest map, and the prompt bullet lists from it:

```
# backend/books.py
BOOKS = {
    "Feeling Good": "feeling_good.pdf",
    ...
}
KNOWN_BOOKS = frozenset(BOOKS)
BOOK_LIST_FOR_PROMPTS = ", ".join(BOOKS)
```

[MEDIUM] 10. Max-retry constant duplicated between the evaluator node and the graph router

`agent/nodes/evaluator.py:93 (attempts >= 3)` and `agent/graph.py:26 (retrieval_attempts >= 3)`

Problem: The retry budget '3' is hardcoded in two files that must stay in sync: the evaluator decides when to write the fallback response, and the router independently decides when to route to END.

Why it matters: If someone changes one and not the other, the graph either ends without a `final_response` (blank answer to the user) or writes the fallback while still retrying.

Recommended fix: Define `MAX_RETRIEVAL_ATTEMPTS = 3` once (`config.py` or `state.py`) and import it in both places. Better still, let only the router own the decision and have the fallback response produced by a dedicated 'give_up' node.

[MEDIUM] 11. History-to-LangChain-message conversion is copy-pasted

agent/nodes/psychologist.py:77-85 and agent/nodes/follow_up.py:37-45

Problem: Both nodes contain the identical loop translating state message dicts into HumanMessage/AIMessage objects and appending the final user message.

Why it matters: Divergence risk: a fix in one (e.g. handling a new role like 'system') will be forgotten in the other.

Recommended fix: Extract a shared helper, e.g. in agent/nodes/_history.py:

```
def to_langchain_messages(messages: list[dict]) -> list:
    out = []
    for m in messages:
        role = m.get("role", "")
        if role == "user":
            out.append(HumanMessage(content=m.get("content", "")))
        elif role in ("assistant", "ai"):
            out.append(AIMessage(content=m.get("content", "")))
    return out
```

[MEDIUM] 12. action_planner swallows every exception silently

agent/nodes/action_planner.py:55-56

Problem: except Exception: state["action_plan"] = [] hides all failures — rate limits, schema validation errors, prompt bugs — with no log line. This node is also the only one that logs nothing at all.

Why it matters: When action plans mysteriously stop appearing in the UI, there is zero signal about why. Silent excepts are the single biggest debugging tax in LLM pipelines.

Recommended fix: Log the exception and narrow the type where possible:

```
except Exception as exc:
    log_warn("ACTION_PLANNER", "structured output failed", repr(exc))
    state["action_plan"] = []
```

[MEDIUM] 13. active_frameworks detection uses fragile substring matching

agent/nodes/psychologist.py:90-91

Problem: A book counts as 'used' only if its exact title string appears in the response. "Thinking Fast and Slow" fails to match if the model writes the official title "Thinking, Fast and Slow" (with comma); partial mentions like 'as Attached explains' match even when no content was used.

Why it matters: The frontend book-sidebar display will randomly miss or over-report sources.

Recommended fix: Match case-insensitively with light normalization (strip commas), or better, have the psychologist output citations in a fixed format like (Book, p.X) and extract them with a regex.

```
import re
cited = set(re.findall(r"\b(Feeling Good|Attached|The Body Keeps the Score|"
    r"Games People Play|Thinking,? Fast and Slow|"
    r"Nonviolent Communication)\b", response.content))
```

[MEDIUM] 14. Guard verdict parsing is fragile and duplicates request initialization

agent/nodes/guard.py:36-38 and 54

Problem: (a) "off_topic" in classification misfires on outputs like 'not off_topic' and silently treats any unexpected output ('I cannot classify this') as relevant. (b) The node re-initializes current_query and retrieval_attempts that api/main.py already set, hiding where state is really born.

Why it matters: LLM classifiers drift; parsing should fail closed and be explicit. Duplicate initialization confuses anyone tracing state flow.

Recommended fix: Compare against exact expected tokens and default to 'relevant' only for the exact token; keep state initialization in one place (the endpoint):

```
verdict = response.content.strip().lower()
is_off_topic = verdict.startswith("off_topic")
```

[MEDIUM] 15. api layer reaches into a private module global

api/main.py:15 (from rag.retriever import _vectorstore)

Problem: The underscore prefix marks _vectorstore as module-private, yet main.py imports it directly for the three /db endpoints.

Why it matters: The privacy convention becomes meaningless and refactoring rag/retriever.py (e.g. lazy init from Finding 4) breaks the API layer.

Recommended fix: Expose a public accessor next to build_retriever():

```
# rag/retriever.py
def get_vectorstore() -> Chroma:
    return _vectorstore
```

[MEDIUM] 16. /db/stats and /db/browse load the entire collection into memory per request

api/main.py:109, api/main.py:162

Problem: _vectorstore.get() with no filter pulls every document and metadata (thousands of chunks) into Python on each call; /db/browse then throws most of it away after filtering one book.

Why it matters: Fine for local debugging, but each request costs tens of MB and O(N) time, and it grows with every ingested book.

Recommended fix: Use Chroma's where-filter and pagination server-side:

```
data = _vectorstore.get(where={"source": book},
                        limit=limit, offset=offset)
```

[MEDIUM] 17. print()-based logger instead of the logging module

agent/logger.py (entire file)

Problem: All node logging goes through print() with hardcoded ANSI codes: no levels, no timestamps, no way to silence it in production or redirect it to a file (log files fill with escape codes).

Why it matters: You cannot tune verbosity per environment, and any log aggregation (Docker, cloud) gets unparseable colored text.

Recommended fix: Keep the nice formatting but route through logging with a level check, e.g. build the colored string and call logging.getLogger("agent").info(...); disable colors when not sys.stdout.isatty().

[MEDIUM] 18. Dead state fields: should_loop and turn_count

agent/state.py:33-34, agent/nodes/follow_up.py:49-50

Problem: follow_up sets should_loop=False and increments turn_count, but no router or node ever reads either field, and turn_count is re-initialized to 0 on every request so it is always 1.

Why it matters: Dead fields mislead readers into thinking there is a conversation loop inside the graph; the loop actually lives in the frontend.

Recommended fix: Delete both fields from AgentState, follow_up.py, and the initial state in api/main.py.

[MEDIUM] 19. Magic numbers and settings scattered across the codebase

MAX_HISTORY=10 (api/main.py:50), k=8 (rag/retriever.py:14), chunk_size=800/overlap=120 (ingest.py:91), max attempts 3 (two files), temperatures (each node), CORS origins, port 8000, model name

Problem: Every tunable lives where it is used, with no single place to see or change the system's configuration.

Why it matters: Tuning retrieval (k, chunk size) or swapping the model requires a scavenger hunt; it also blocks environment-specific overrides.

Recommended fix: Add a backend/config.py with named constants (optionally pydantic-settings so env vars can override), and import from it everywhere.

Low-Severity Findings

[LOW] 20. Bare string literals used as comments

`api/main.py:28-33`, `agent/nodes/guard.py:43-48`, `agent/nodes/retriever.py:28-31`,
`rag/ingest.py:21-27` and `30-36`

Problem: Triple-quoted strings placed after statements (below `add_middleware`, inside `run()`, after list literals) are not docstrings — they are no-op expression statements that allocate a string and discard it.

Why it matters: It misleads readers about what a docstring is, and linters (ruff B018) flag it as a useless expression.

Recommended fix: Convert them to regular `#` comments above the code they describe.

[LOW] 21. Mid-function comments at column zero, including a typo

`agent/nodes/retriever.py:87, 98`; `agent/nodes/psychologist.py:86, 89` (`'booksider'`)

Problem: Several comments sit at indentation level 0 inside indented blocks, breaking visual flow; `psychologist.py:89` says `'booksider display'` (likely `'book sidebar'`).

Why it matters: Minor readability issue that also trips auto-formatters.

Recommended fix: Indent comments with the code they annotate; fix the typo.

[LOW] 22. Inconsistent naming between files

`psychologist.py` uses `c/msg/messages`; `follow_up.py` uses `message/conversation`; `retriever.py` uses `doc/chunk`

Problem: The same concepts (a chunk, a message, a message list) have different names across nodes.

Why it matters: Slows down reading; the files are otherwise nearly identical in shape.

Recommended fix: Pick one vocabulary (chunk, message, history) and apply it across all nodes.

[LOW] 23. Evaluator log always appends an ellipsis

`agent/nodes/evaluator.py:68`

Problem: The query preview appends `'...'` unconditionally, even when the message is shorter than 100 characters — unlike `guard.py:41` which does it correctly.

Why it matters: Cosmetic log accuracy.

Recommended fix: Reuse `guard.py`'s conditional-ellipsis pattern (another argument for a shared logging helper).

[LOW] 24. Message role vocabulary handled inconsistently

`psychologist.py:83`, `follow_up.py:43` accept `'assistant'` and `'ai'`; `api` Message model does not restrict roles

Problem: Nodes accept both `'assistant'` and `'ai'` roles, while unknown roles are silently dropped from the conversation.

Why it matters: A frontend typo in the role field silently deletes history instead of failing loudly.

Recommended fix: Restrict roles in the Pydantic Message model (`Literal['user','assistant']`) and drop the `'ai'` alias in the nodes.

[LOW] 25. MAX_HISTORY slicing can break user/assistant pairing

api/main.py:63

Problem: messages[-10:] can start the window with an assistant message, giving the LLM a history that opens with an answer to an invisible question.

Why it matters: Mildly degrades response quality on long conversations.

Recommended fix: After slicing, drop leading non-user messages (or slice by conversational turns instead of raw messages).

[LOW] 26. reload=True and 0.0.0.0 in the default entry point

start.py:15

Problem: The single entry point hardcodes dev settings: hot-reload and binding to all interfaces.

Why it matters: reload spawns a file-watcher subprocess (surprising in production) and 0.0.0.0 exposes the API to the local network.

Recommended fix: Gate via env: `uvicorn.run(..., host=os.getenv('HOST', '127.0.0.1'), reload=os.getenv('DEV')=='1')`.

[LOW] 27. messages typed as List[dict] in AgentState

agent/state.py:24

Problem: The message shape (`{'role': str, 'content': str}`) is implicit; every consumer re-discovers it with `.get()` calls.

Why it matters: Type checkers cannot catch typos like `message['contnet']`.

Recommended fix: Define a small TypedDict and use `List[ChatMessage]` (mirrors the API-layer Pydantic model).

[LOW] 28. No tests

backend/ (no tests directory)

Problem: There are no unit or integration tests. The graph routing (`route_after_evaluation` edge cases), the chunk filters (`_is_content`, `_filter_chunks`, `_has_foreign_reference`), and the `/chat` validation are all pure or cheap to test without hitting an LLM.

Why it matters: Refactors like the ones in this review are risky without a safety net; the filter functions in particular encode subtle business rules that regress easily.

Recommended fix: Start with pytest on the pure functions: `_is_content` junk cases, `_filter_chunks` foreign-marker cases, both graph routers, and a `/chat 422` test via FastAPI's `TestClient`. None require API keys once Finding 4 (lazy init) is done.

[LOW] 29. Trailing whitespace / stray indentation at end of ingest.py

rag/ingest.py:134

Problem: The final line after `ingest()` contains stray indentation.

Why it matters: Trivial hygiene; some tools warn on it.

Recommended fix: Run a formatter (`ruff format` or `black`) over the backend to normalize this and the aligned-colon dict style consistently.

What the Code Does Well

- **Clean layering.** `api / agent / agent.nodes / rag` is exactly the right decomposition; each node is a single-responsibility file with one `run()` entry point.
- **Excellent docstrings.** Nearly every function explains not just what but why — rare and valuable.
- **Thoughtful ingestion filtering.** The junk-pattern and index-page detection in `ingest.py:_is_content` shows real attention to RAG data quality, as does the foreign-book-reference filter in the retriever node.
- **Structured output for the action planner.** Using `with_structured_output` + a Pydantic schema is the right tool, with a fallback path.
- **Secrets hygiene.** `.env` is gitignored with a `.env.example` template; the factory fails fast with a clear message when the key is missing.
- **Bounded retry loop.** The evaluator/query_builder retry cycle has a hard cap and a graceful ask-the-user fallback rather than looping forever.
- **Grounding and citation prompt design.** The psychologist system prompt (grounding rule, demonstration rule, citation whitelist) is well beyond typical RAG prompts.

Suggested Priority Order

#	Work item	Effort	Payoff
1	Findings 1, 2 — request validation + SSE error event	~30 min	Stops crashes and frontend hangs immediately
2	Finding 3 — absolute paths for <code>chroma_db</code> and <code>books</code>	~20 min	Eliminates the silent empty-DB failure mode
3	Findings 8 — fix <code>requirements.txt</code>	~5 min	Fresh installs work
4	Findings 12, 14 — log the planner exception, harden guard clauses	15 min	Debuggability
5	Findings 9, 10, 19 — <code>config.py</code> + <code>books.py</code> single source of truth	1 h	Kills the duplication class of bugs
6	Findings 4, 5, 7 — lazy init, partial state updates, async nodes	2-3 h	Testability, correctness under load; do together as one ref
7	Findings 11, 13, 15-18, 20-29	as time allows	Polish and hygiene; finish with a formatter pass and first t

Overall: this is a solid, readable codebase whose issues are concentrated in operational robustness (validation, error handling, paths) rather than design. Working through items 1-5 above (~2 hours) removes every high-severity risk.